Layouts: Organizando a Interface

Tkinter e CustomTkinter

Prof. Jelcimar Souza

IFES Cachoeiro de Itapemirim

14 de outubro de 2026

Objetivos da aula

Ao final desta aula, você será capaz de:

- escolher entre `pack()`, `grid()` e `place()`;
- organizar widgets em linhas, colunas e regiões;
- agrupar elementos usando `Frame` e `CTkFrame`;
- criar interfaces que se ajustam ao redimensionamento da janela;
- identificar e corrigir erros comuns de layout.

O que é um gerenciador de layout?

Conceito

Um gerenciador de layout define como os widgets são posicionados e como ocupam o espaço disponível dentro de uma janela ou contêiner.

- Evita posicionar cada elemento manualmente.
- Ajuda a manter a interface organizada quando a janela muda de tamanho.
- Facilita a manutenção e a inclusão de novos componentes.

Importante: o layout é aplicado dentro do contêiner ao qual o widget pertence.

Os três gerenciadores principais

Gerenciador	Como funciona	Uso comum
`pack()`	Organiza widgets em blocos, usando lados e preenchimento.	Barras, botões empilhados e layouts simples.
`grid()`	Organiza widgets em uma grade de linhas e colunas.	Formulários, telas de cadastro e calculadoras.
`place()`	Posiciona por coordenadas absolutas ou relativas.	Casos específicos com posição controlada.

Regra essencial

Não use `pack()` e `grid()` no mesmo contêiner. É possível usá-los em contêineres diferentes.

1. Usando pack()

```
import tkinter as tk

janela = tk.Tk()
janela.title("Exemplo com pack")
janela.geometry("360x240")

tk.Label(janela, text="Cabeçalho", bg="lightblue").pack(
    side="top", fill="x", padx=10, pady=10
)
tk.Button(janela, text="Salvar").pack(pady=5)
tk.Button(janela, text="Cancelar").pack(pady=5)

janela.mainloop()
```

- `side`: define o lado de posicionamento.
- `fill`: permite preencher horizontalmente ou verticalmente.
- `padx` e `pady`: criam espaçamento externo.

pack(): preenchimento e expansão

```
import tkinter as tk

janela = tk.Tk()
janela.geometry("420x260")

painel = tk.Frame(janela, bg="white")
painel.pack(fill="both", expand=True, padx=15, pady=15)

tk.Label(painel, text="Área principal", bg="lightblue").pack(
    fill="both", expand=True
)
tk.Button(janela, text="Fechar", command=janela.destroy).pack(pady=8)

janela.mainloop()
```

`fill="both"` permite ocupar largura e altura; `expand=True` permite receber espaço extra quando a janela aumenta.

2. Usando grid()

```
import tkinter as tk

janela = tk.Tk()
janela.title("Cadastro")

tk.Label(janela, text="Nome:").grid(row=0, column=0, padx=8, pady=8, sticky="e")
nome = tk.Entry(janela, width=28)
nome.grid(row=0, column=1, padx=8, pady=8)

tk.Label(janela, text="E-mail:").grid(row=1, column=0, padx=8, pady=8, sticky="e")
email = tk.Entry(janela, width=28)
email.grid(row=1, column=1, padx=8, pady=8)

tk.Button(janela, text="Enviar").grid(row=2, column=0, columnspan=2, pady=12)
janela.mainloop()
```

`row` indica a linha; `column`, a coluna; `columnspan` permite ocupar mais de uma coluna; `sticky` controla o alinhamento.

grid(): adaptando linhas e colunas

```
# Depois de criar a janela e os widgets:
janela.columnconfigure(1, weight=1)
janela.rowconfigure(2, weight=1)

campo = tk.Entry(janela)
campo.grid(row=0, column=1, sticky="ew", padx=8, pady=8)

area = tk.Text(janela, height=5)
area.grid(row=2, column=0, columnspan=2,
          sticky="nsew", padx=8, pady=8)
```

- `weight=1` permite que a linha ou coluna receba espaço extra.
- `sticky="ew"` estica horizontalmente.
- `sticky="nsew"` estica nas quatro direções.

3. Usando place()

```
import tkinter as tk

janela = tk.Tk()
janela.geometry("400x250")

tk.Label(janela, text="Painel de controle").place(x=130, y=25)
tk.Button(janela, text="Ligar").place(x=90, y=90, width=90)
tk.Button(janela, text="Desligar").place(x=210, y=90, width=90)

tk.Label(janela, text="Centralizado").place(
    relx=0.5, rely=0.75, anchor="center"
)
janela.mainloop()
```

- `x` e `y`: coordenadas em pixels.
- `relx` e `rely`: posição relativa ao contêiner, de 0 a 1.

Limitação: posições fixas podem ficar inadequadas quando a janela é redimensionada.

Qual devo escolher?

pack()	grid()	place()
Empilhar	**Grade**	**Coordenadas**
• Menu lateral	• Formulários	• Elementos sobrepostos
• Botões em sequência	• Calculadoras	• Posição específica
• Cabeçalho e rodapé	• Campos alinhados	• Protótipos simples

Para a maioria dos formulários, comece com `grid()`; para sequências simples, experimente `pack()`.

Organizando a interface com Frame

```
import tkinter as tk

janela = tk.Tk()
janela.title("Tela organizada")
janela.geometry("420x260")

cabecalho = tk.Frame(janela, bg="#dceeff")
cabecalho.pack(fill="x")
tk.Label(cabecalho, text="Sistema de Cadastro",
         bg="#dceeff", font=("Arial", 16, "bold")).pack(pady=12)

conteudo = tk.Frame(janela, padx=15, pady=15)
conteudo.pack(fill="both", expand=True)
tk.Label(conteudo, text="Nome:").grid(row=0, column=0, sticky="w")
tk.Entry(conteudo).grid(row=0, column=1, padx=8, pady=6)
tk.Button(conteudo, text="Salvar").grid(row=1, column=1, sticky="e")

janela.mainloop()
```

Frames: por que dividir a janela?

- Cada `Frame` funciona como um contêiner independente.
- Dentro de um frame, você pode usar `pack()` ou `grid()` conforme a necessidade.
- Dividir a tela em cabeçalho, conteúdo e rodapé deixa o código mais legível.
- O mesmo princípio existe no CustomTkinter com `CTkFrame`.

Exemplo de organização

Janela principal $\rightarrow$ cabeçalho + área de formulário + área de ações.

Layouts no CustomTkinter

```
import customtkinter as ctk

ctk.set_appearance_mode("system")
app = ctk.CTk()
app.title("Cadastro moderno")
app.geometry("420x300")

formulario = ctk.CTkFrame(app)
formulario.pack(fill="both", expand=True, padx=20, pady=20)

ctk.CTkLabel(formulario, text="Nome").grid(
    row=0, column=0, padx=10, pady=10, sticky="e"
)
ctk.CTkEntry(formulario, placeholder_text="Digite seu nome").grid(
    row=0, column=1, padx=10, pady=10, sticky="ew"
)
formulario.grid_columnconfigure(1, weight=1)
app.mainloop()
```

Os conceitos de layout são os mesmos; o CustomTkinter oferece widgets com aparência moderna.

Erros comuns e como evitar

- **Misturar pack e grid no mesmo contêiner:** escolha um gerenciador para aquele contêiner.
- **Esquecer espaçamentos:** use `padx` e `pady` para melhorar a leitura.
- **Janela não se adapta:** configure `weight`, `sticky`, `fill` e `expand` quando apropriado.
- **Usar place para tudo:** posições fixas dificultam a adaptação a tamanhos diferentes.
- **Código difícil de manter:** agrupe widgets relacionados em frames.

Atividade prática: tela de cadastro

Desafio: crie uma tela de cadastro de estudante com Tkinter ou CustomTkinter.

1. Adicione um título e campos para nome, e-mail e curso.
2. Organize os campos com `grid()`.
3. Coloque os campos dentro de um `Frame`.
4. Crie botões **Salvar** e **Limpar**.
5. Redimensione a janela e observe o comportamento do layout.

Desafio extra

Faça o campo de e-mail crescer horizontalmente quando a janela for ampliada.

Resumo e referências

- `pack()`: organiza widgets em blocos.
- `grid()`: organiza widgets em linhas e colunas.
- `place()`: posiciona por coordenadas.
- `Frame`/`CTkFrame`: divide a interface em áreas menores.
- `weight`, `sticky`, `fill` e `expand`: ajudam na adaptação do layout.

Referências

- `docs.python.org/3/library/tkinter.html`
- `customtkinter.tomschimansky.com`

Obrigado!